UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

BASES DE DATOS II

Unidad Temática N°4:
Bases de Datos en Columnas

Teórico
2° Año – 3° Cuatrimestre

CC BY ND NC

Introducción a las bases de datos basadas columnares 3

Arquitectura de las Bases de Datos Columnares. 3
Ventajas de las Bases de Datos Columnares 4
Desventajas de las Bases de Datos Columnares 5
Comparación con Bases de Datos Relacionales y Otras NoSQL 5
Cassandra: Implementación de Base de Datos Columnares 7
Modelo de Columnas en Cassandra 7
Consistencia Eventual en Cassandra 9
Niveles de Consistencia 10

Casos de Uso de Cassandra 10

Análisis de Series Temporales 10
Datos Distribuidos 11
Conclusión 11

BIBLIOGRAFÍA 13

Introducción a las bases de datos basadas columnares

Las bases de datos columnares (o bases de datos NoSQL columnares) son una categoría de bases de datos NoSQL diseñadas para almacenar, recuperar y gestionar datos en columnas en lugar de filas. En una base de datos relacional tradicional, los datos se organizan en filas y columnas, donde cada fila representa un registro completo y cada columna un atributo del mismo.

En contraste, en una base de datos columnar, los datos se almacenan de manera que las columnas se organizan y gestionan de manera separada, lo que permite un acceso más eficiente a los datos, especialmente para consultas que implican grandes volúmenes de datos de una sola columna..

Arquitectura de las Bases de Datos Columnares.

En lugar de almacenar las filas de datos de manera tradicional (como en bases de datos relacionales), las bases de datos columnares almacenan los datos por columna. Es decir, los valores de cada columna se agrupan juntos, en vez de almacenar todos los valores de una fila juntos.

Modelo de almacenamiento:

En una base de datos columnares, los datos de cada columna se agrupan y se almacenan en bloques contiguos de almacenamiento físico. Esto puede proporcionar mejoras significativas en el rendimiento para ciertas consultas que solo requieren acceder a algunas columnas y no a todas las filas completas.

Estructura de almacenamiento:

- **Familia de columnas** (Column Family): Similar a las tablas de las bases de datos relacionales, pero aquí las familias de columnas pueden contener diferentes columnas, y cada fila puede tener un conjunto diferente de columnas. Las familias de columnas son colecciones lógicas de columnas, y una base de datos columnar puede tener múltiples familias de columnas.
- **Filas** (Rows): Aunque los datos se almacenan de manera columnar, cada conjunto de valores pertenece a una fila específica identificada por una clave primaria.
- **Columnas**: En una base de datos columnar, cada columna está compuesta por un conjunto de valores. Por ejemplo, si se tiene una columna llamada

"Edad", todos los valores de la columna "Edad" en una tabla se almacenan de manera contigua. Esta estructura es ventajosa cuando se requiere hacer lecturas rápidas sobre columnas específicas, ya que no se tiene que leer toda la fila para obtener la información de esa columna.

Ventajas de las Bases de Datos Columnares

Las bases de datos columnares ofrecen varias ventajas que las hacen especialmente adecuadas para ciertos tipos de aplicaciones y cargas de trabajo.

- **Lecturas rápidas de columnas específicas**: Dado que las columnas se almacenan de manera contigua, las bases de datos columnares son muy eficientes cuando se realizan consultas que solo acceden a un pequeño subconjunto de columnas. Por ejemplo, si se quiere obtener solo los valores de una columna específica (como la "edad" de una lista de personas), la base de datos solo necesita leer la columna de "edad", lo que puede mejorar enormemente el rendimiento de la consulta.
- **Compresión eficiente**: Dado que los valores de una columna suelen ser similares entre sí, es posible aplicar algoritmos de compresión eficientes. Esto reduce significativamente el espacio de almacenamiento y mejora el rendimiento de las consultas.
- **Escalabilidad horizontal**: Las bases de datos columnares como Cassandra están diseñadas para escalar horizontalmente, lo que significa que se pueden agregar más nodos a medida que crece el volumen de datos sin degradar el rendimiento. Esto es crucial en entornos donde los volúmenes de datos crecen rápidamente.
- **Distribución de datos eficiente**: La capacidad de distribuir datos en varios nodos de manera eficiente permite que las bases de datos columnares gestionen grandes volúmenes de datos distribuidos geográficamente, como en los casos de uso de Big Data.
- **Esquemas flexibles**: En lugar de imponer una estructura rígida, las bases de datos columnares permiten definir esquemas de manera dinámica, lo que facilita la modificación de la estructura sin interrumpir las operaciones.
- **Análisis de grandes volúmenes de datos**: Son especialmente eficaces en entornos de análisis de datos masivos, como en los casos de uso de análisis de logs, análisis de series temporales, análisis de tendencias, entre otros.

Desventajas de las Bases de Datos Columnares

Aunque las bases de datos columnares tienen muchas ventajas, también existen algunas desventajas y limitaciones que deben considerarse al elegir este tipo de sistema.

- **Escrituras menos eficientes**: Las bases de datos columnares pueden no ser tan eficientes para cargas de trabajo que requieren un alto volumen de escrituras. Esto se debe a que las escrituras en una base de datos columnar generalmente implican más operaciones, ya que los datos de cada columna deben almacenarse en bloques separados.
- **No aptas para todas las aplicaciones**: Las bases de datos columnares son particularmente útiles para aplicaciones de lectura intensiva donde solo se consultan unas pocas columnas, pero pueden no ser ideales para sistemas de transacciones complejas o aquellos que requieren actualizaciones frecuentes de todas las columnas de una fila.
- **Modelo de datos no relacional**: Aunque esto no es necesariamente una desventaja, las bases de datos columnares no son adecuadas para todos los tipos de aplicaciones. Las aplicaciones que dependen de relaciones complejas entre diferentes tablas pueden no beneficiarse tanto de este modelo.

Comparación con Bases de Datos Relacionales y Otras NoSQL

Las bases de datos columnares se destacan frente a otros tipos de bases de datos NoSQL (como las bases de datos clave-valor o de documentos) y bases de datos relacionales, especialmente en términos de escalabilidad, flexibilidad y rendimiento para consultas de lectura.

Frente a bases de datos relacionales:

- **Ventajas**: Las bases de datos columnares son mucho más eficientes cuando se realizan consultas sobre grandes volúmenes de datos de columnas específicas, como en el análisis de Big Data. Además, son mucho más escalables, lo que las hace adecuadas para aplicaciones distribuidas a gran escala**.**
- **Desventajas:** Las bases de datos relacionales son más adecuadas para aplicaciones que requieren integridad referencial, transacciones ACID y operaciones complejas entre múltiples tablas (joins, subconsultas, etc.).

Base de datos relacional

Id	Nombre	Direccion	E_mail	Tel
1	Ana	San Juan 3111	ana@sumail.com	3511112222
2	Juan	Avellaneda 2555		3519999888
3	Alex	Belgrano 9999	alex@jmail.com	

Base de datos columnar

Id	Columna	Valor
1	Nombre	Ana
1	Direccion	San Juan 3111
1	E_mail	ana@sumail.com
1	Tel	3511112222
2	Nombre	Juan
2	Direccion	Avellaneda 2555
2	Tel	3519999888
3	Nombre	Alex
3	Direccion	Belgrano 9999
3	E_mail	alex@jmail.com

Se puede pueden observar tres columnas en la tabla de la base de datos columnar, cada fila de la BD relacional se transforma en múltiples filas lógicas de la columnar.

El id se denomina row key el cual es único para cada registro de la base de datos columnar.

Cada atributo se identifica por un nombre de columna y tiene un valor asociado.

Los atributos se pueden modificar dinámicamente sin cambiar la estructura de la base de datos columnar a diferencia de una BD relacional.

BD Relacionales	BD Columnares
Datos en filas y columnas	Dato en columnas
Soporta SQL	BD NoSQL
Consultas complejas con agrupaciones joins, etc.	Operaciones básicas
Estructura normalizada para minimizar redundancia	Almacenamiento desnormalizado para minimizar búsquedas
ACID compliant	ACID compliant a nivel fila

Frente a bases de datos clave-valor y de documentos:

- **Ventajas:** Las bases de datos columnares pueden ser más eficaces cuando se trata de consultas que involucran grandes volúmenes de datos organizados en columnas y cuando las consultas son más complejas que una simple recuperación de valor clave. La organización de los datos por columnas también facilita la compresión y el almacenamiento eficiente.
- **Desventajas:** Las bases de datos clave-valor y de documentos pueden ser más adecuadas cuando la estructura de los datos es más flexible o cuando se necesitan operaciones rápidas de lectura y escritura en valores no estructurados.

Cassandra: Implementación de Base de Datos Columnares

Cassandra es una de las bases de datos más conocidas que implementa el modelo de columnares de manera eficiente. Está diseñada para manejar grandes volúmenes de datos distribuidos en múltiples nodos, y su arquitectura distribuida está optimizada para la alta disponibilidad y la tolerancia a fallos.

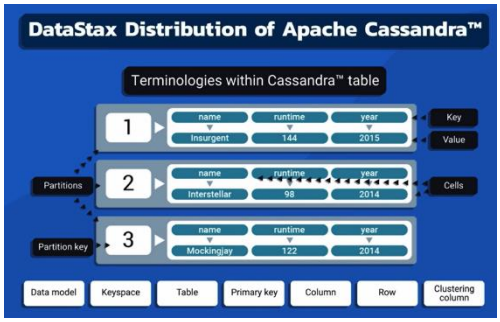
- **Modelo de datos**: En Cassandra, los datos se almacenan en familias de columnas, que son equivalentes a las tablas en bases de datos tradicionales. Sin embargo, dentro de una familia de columnas, cada fila puede tener un número diferente de columnas, lo que ofrece gran flexibilidad.
- **Uso de Familia de Columnas**: Las Familia de Columnas son el contenedor lógico de las columnas. Aunque el modelo de columnas es flexible, el diseño de los datos debe optimizarse para mejorar la eficiencia en las consultas.
- **Esquemas dinámicos**: A diferencia de las bases de datos relacionales, donde el esquema debe definirse de antemano, en Cassandra el esquema es flexible, lo que significa que las columnas pueden agregarse o eliminarse sin interrumpir el funcionamiento del sistema.
- **Ventajas**: Es capaz de manejar petabytes de datos en clústeres distribuidos de miles de nodos, y puede garantizar alta disponibilidad y escalabilidad horizontal.
- **Escalabilidad horizontal**: Los nodos se agregan fácilmente sin que sea necesario redistribuir los datos.
- **Alta disponibilidad y tolerancia a fallos**: Utiliza un modelo de replicación para asegurar que los datos estén disponibles incluso si hay fallos en algunos nodos.
- **Modelo de datos flexible**: Cassandra no impone una estructura rígida de datos como las bases de datos relacionales. Los esquemas pueden ser modificados dinámicamente.

Modelo de Columnas en Cassandra

Cassandra utiliza un modelo basado en columnas en lugar de filas, lo que significa que los datos se organizan en familias de columnas en lugar de tablas

tradicionales. Para ir entendiendo cómo Cassandra almacena datos se enuncian aquí la terminología básica de su uso:

- **Data Model**: (modelo de datos) es un modelo abstracto para organizar los elementos de los datos. Los modelos de datos varían según el tipo, la capacidad y el propósito de la base de datos. El tipo de modelo de datos depende de la capacidad de la base de datos. Para Cassandra, el modelo de datos se basa en las consultas que se deseen realizar.



- **Keyspace**: Un keyspace en Cassandra es un contenedor lógico para un conjunto de familias de columnas. Se puede pensar como una base de datos que contiene tablas (Familia de Columnas). almacena tablas y datos de replicación, en otras palabras, es un contenedor para la replicación.

- **Column Family**: En Cassandra, los datos se organizan en "Familias de Columnas", que son equivalentes a las tablas en una base de datos relacional, es decir, es una combinación de filas y columnas. Sin embargo, a diferencia de las tablas tradicionales, las familias de columnas no requieren un esquema rígido de columnas. Las tablas o column family están contenidas dentro del keyspace

- **Rows**: Una fila en Cassandra está compuesta por una clave primaria y una serie de columnas. Cada fila puede tener un número variable de columnas, y cada columna tiene un valor y un timestamp.

- **Columns**: Una columna tiene una clave, un valor y un timestamp que indica cuándo se insertó o actualizó el valor. Esto le da a Cassandra una ventaja en términos de eficiencia para almacenar y recuperar grandes volúmenes de datos, especialmente cuando se trabaja con grandes cantidades de datos temporales o de series.

- **Partitions:** Como base de datos columnar, Cassandra almacena y recupera datos basándose en particiones. Una partición son filas de datos sin procesar que se almacenan en un nodo de la tabla de datos según la estrategia de partición. Cada fila de la partición consta de pares clave-valor que Cassandra almacena. Dentro de cada partición hay filas y columnas.

- **Partition key:** Una clave de partición es la primera parte de la clave principal y determina en qué nodo del clúster se almacenarán los datos. Una clave de

partición es una columna a la izquierda del gráfico que indica uno, dos o tres, respectivamente.

- **Primary Key:** una Clave principal es la parte más importante del modelo de datos y cumple dos funciones: primero, garantiza la unicidad de los datos y, segundo, define la ubicación del registro en el clúster. Esto facilita el acceso a los datos en el modelo.

Concepto	Equivalente / Rol
Keyspace	Base de datos
Table (Column Family)	Tabla
Partition Key	Clave que determina dónde se guarda la fila (distribución)
Clustering Column	Orden de los datos dentro de la partición
Primary Key	Partition Key + Clustering Column(s)
Row	Conjunto de columnas dentro de una partición
Column	Atributo con nombre y valor (flexible por fila)

Consistencia Eventual en Cassandra

Concepto de Consistencia Eventual

La consistencia eventual es un principio de sistemas distribuidos en el que las actualizaciones en un sistema no se propagan inmediatamente a todos los nodos. En lugar de garantizar que todas las copias de los datos estén inmediatamente sincronizadas, Cassandra permite que las lecturas puedan devolver valores inconsistentes temporalmente, pero con la garantía de que finalmente se alcanzará la consistencia.

Fundamentos

En Cassandra, cada operación de escritura se replica en varios nodos del clúster. Sin embargo, no todos los nodos deben estar actualizados de inmediato. Cassandra permite que los nodos se sincronicen en un período de tiempo, y durante ese tiempo, pueden ocurrir lecturas inconsistentes.

Ventajas:

- **Alta disponibilidad**: El sistema siempre puede seguir funcionando incluso si algunos nodos están fuera de servicio.
- **Escalabilidad**: Dado que no se requiere que todos los nodos estén sincronizados instantáneamente, se pueden agregar nodos fácilmente para distribuir la carga.

Desventajas:

- **Posibles lecturas inconsistentes**: Durante el proceso de replicación, los nodos pueden devolver datos diferentes, lo que requiere mecanismos adicionales para manejar la inconsistencia temporal.

Niveles de Consistencia

Cassandra permite a los desarrolladores elegir entre varios niveles de consistencia para las operaciones de lectura y escritura, lo que puede influir en el rendimiento y la consistencia del sistema. Los niveles más comunes son:

- **ONE**: La operación se considera exitosa cuando uno de los nodos ha confirmado la escritura o lectura.
- **QUORUM**: La operación se considera exitosa cuando la mayoría de los nodos (más de la mitad) han confirmado la operación.
- **ALL**: Todos los nodos deben confirmar la operación.

Casos de Uso de Cassandra

Análisis de Series Temporales

Cassandra es especialmente eficiente cuando se trabaja con series temporales debido a su capacidad para almacenar datos en formato columnar y su arquitectura distribuida.

Características del análisis de series temporales:

Los datos a menudo están indexados por timestamp, lo que facilita su consulta. ¿Qué es timestamp? Un timestamp en Cassandra es una marca de tiempo

que se genera cada vez que se escriben datos en la base de datos. Se utiliza para resolver conflictos en los cambios que se realizan en el mismo valor.

Los datos tienden a crecer rápidamente, lo que hace que la escalabilidad horizontal de Cassandra sea crucial.

Ejemplo de uso:

Monitoreo de sistemas: En un sistema de monitoreo de infraestructura de TI, los sensores de temperatura pueden registrar lecturas de manera continua. Cassandra puede manejar el volumen de datos a lo largo del tiempo y realizar consultas rápidas por hora, día o mes.

Big Data: Las series temporales son un componente común en el análisis de Big Data, donde las empresas necesitan procesar grandes cantidades de datos generados a través del tiempo.

Datos Distribuidos

Cassandra es ideal para almacenar datos distribuidos debido a su naturaleza descentralizada y su capacidad de replicación de datos entre varios nodos y centros de datos.

Distribución de datos en Cassandra:

Los datos se replican en nodos distribuidos en diferentes centros de datos, lo que proporciona alta disponibilidad y tolerancia a fallos.

Cassandra utiliza un enfoque de "hashing" para distribuir los datos entre los nodos, asegurando que los datos estén balanceados en todo el clúster. ¿Qué es hashing? El hashing consistente en Cassandra es una técnica que permite distribuir datos en un clúster para reducir la reorganización de datos cuando se agregan o eliminan nodos

Conclusión

Las bases de datos columnares son una excelente opción para almacenar y gestionar grandes volúmenes de datos distribuidos, especialmente cuando las consultas están orientadas a leer grandes cantidades de datos de unas pocas columnas. Cassandra, como implementación distribuida de base de datos columnar, se destaca por su capacidad para manejar datos masivos con alta disponibilidad y escalabilidad horizontal.

Sin embargo, su arquitectura no es la más adecuada para todas las aplicaciones, especialmente aquellas que requieren operaciones complejas de escritura o relaciones entre tablas.

BIBLIOGRAFÍA

Amazon Web Services (2024) *¿Cuál es la diferencia entre Cassandra y MongoDB?* Extraido de: https://aws.amazon.com/es/compare/the-difference-between-cassandra-and-mongodb/

DataStax (2025) *DataStax Academy. The place for Apache Cassandra® self-paced classes and certifications*. Disponible en: https://www.datastax.com/academy

Fernandez O (2025) *Apache Cassandra: Introducción*. AprenderBigData.com Extraido de: https://aprenderbigdata.com/introduccion-apache-cassandra/

INOS (2020) *Apache Cassandra: gestión distribuida de grandes bases de datos* Equipo editorial de IONOS. Extraido de: https://www.ionos.com/es-us/digitalguide/hosting/cuestiones-tecnicas/apache-cassandra/

McFadin, J., & Rickard, E. (2022). Managing Cloud-Native Data on Kubernetes: Apache Cassandra Operations in Kubernetes. O'Reilly Media.

Requena Mesa A (2019) Qué es Apache Cassandra. Open Webinars Extraído de: https://openwebinars.net/blog/que-es-apache-cassandra/

Sitaram Chamarty (2020). *Practical Cassandra: A Developer's Approach*. Apress

Strickland, A. (2020). *Cassandra: The Definitive Guide*. O'Reilly Media.

The Apache Software Foundation (2025) *Cassandra Documentation.* Extraido de: https://cassandra.apache.org/doc/latest/

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.